



POS — Project State

Información General

Proyecto: Personal Operating System (POS)

Versión: v0.1

Estado: Domain Model

Fase actual: 05. System Architecture

Objetivo actual: Consolidar el modelo conceptual del dominio definiendo relaciones, ownership, transformaciones, invariantes y ciclos de vida antes de diseñar la arquitectura del sistema.

Última actualización: 10/7/2026

Estado General

Área	Estado
Visión	✓
Foundation	✓
Domain Discovery	✓
Ubiquitous Language	✓
Domain Model	✓
System Architecture	⌚ Pendiente
Information Flow	⌚ Pendiente
Conceptual Data Model	⌚ Pendiente
Governance	⌚ Pendiente
ADRs	⌚ Pendiente
Glossary	⌚ Pendiente
Future Ideas	⌚ Pendiente

Objetivo del Proyecto

Diseñar un Personal Operating System independiente de cualquier herramienta, capaz de organizar trabajo, conocimiento y capacidades mediante una arquitectura conceptual sólida, escalable y mantenible.

Filosofía

- Human in the Loop
- Architecture Before Implementation
- Specs First
- Tool Agnostic
- Docs as Code
- Modularidad
- Escalabilidad
- Simplicidad
- Evolución continua
- Reproducibilidad

Descubrimientos Fundamentales Confirmados

- El POS organiza transformaciones, no información.

Hipótesis

- El dominio es process-centric.
- Operation será el principal Aggregate Root operativo.

- El trabajo está compuesto principalmente por operaciones recurrentes.
 - Los proyectos representan trabajo temporal.
 - El conocimiento se construye; no se almacena.
 - El sistema debe reducir la carga cognitiva.
 - La arquitectura debe ser independiente de cualquier herramienta.
 - Las entidades evolucionan mediante procesos.
- El sistema estará compuesto por múltiples motores especializados.
 - El conocimiento y los activos evolucionarán como capacidades persistentes.

Ubiquitous Language

Entities

Estructurales

- Context
- Workspace

Operativas

- Operation
- Project
- Task

Activos

- Resource
- Knowledge
- Asset
- Deliverable
- Idea
- Inbox

Capacidades y Medios

- Discipline
- Tool

Processes

- Capture
- Process
- Execute
- Learn
- Build
- Deliver
- Review
- Archive

States

- Captured
- Processed
- Planned
- In Progress
- On Hold
- Completed
- Archived
- Discarded

Domain Model

Aggregate Roots

- Context
- Workspace
- Operation
- Project

Ownership

Context

- Workspace
- Asset

Workspace

- Inbox
- Idea

Shared Entities

- Knowledge
- Resource
- Discipline
- Tool

Relationship Types

- Contains
- Uses
- Produces
- Transforms

- Operation
- Project

Operation

- Task
- Deliverable

Project

- Task
- Deliverable

Entity Transformations

Estado: v0.1 definido.

Domain Invariants

Estado: v0.1 definido.

Lifecycles

Estado: v0.1 definido.

Arquitectura

Arquitectura conceptual

Estado: 🟡 En definición.

Engines (v0.1)

El POS organiza sus capacidades mediante cuatro Engines conceptuales.

Cada Engine agrupa uno o más **Bounded Contexts**, responsables de encapsular un modelo de dominio consistente.

Motores

Estado actual:

Engine	Propósito	Bounded Contexts
Capture Engine	Incorporar información al sistema	Capture
Operations Engine	Gestionar el trabajo operativo	Operations
Knowledge Engine	Transformar información en conocimiento y activos	Knowledge, Asset
System Engine	Mantener la arquitectura y el funcionamiento del POS	System

Componentes principales

Pendiente.

Bounded Contexts

Pendiente.

Eventos del dominio

Pendiente.



Information Flow

Estado: ⌚ Pendiente.
Definirá:

- Flujo de entidades.
- Flujo de procesos.
- Flujo de transformaciones.
- Flujo de conocimiento.



Conceptual Data Model

Estado: ⌚ Pendiente.
Pendiente definir:

- Entidades finales.
- Relaciones.
- Cardinalidades.
- Value Objects.
- Identidad.
- Agregados.



Governance

Estado: ⌚ Pendiente.
Pendiente definir:

- Versionado.
- Convenciones.
- Naming.
- Gestión documental.
- Ciclo de cambios.
- Revisión arquitectónica.



ADRs

Estado: ⌚ Pendiente.
ADRs registrados:

- Architecture Before Implementation.
- Tool Agnostic.
- Specs First.
- Operation como núcleo del dominio (hipótesis).
- El POS organiza transformaciones.

Riesgos Arquitectónicos Confirmados

- ✓ No acoplar el modelo a una herramienta.
- ✓ Evitar convertir el sistema en un gestor de tareas.
- ✓ Mantener un lenguaje de dominio consistente.

Abiertos

- ⚠ Validar Aggregate Roots definitivos.
- ⚠ Confirmar motores del sistema.
- ⚠ Definir Bounded Contexts.
- ⚠ Formalizar eventos del dominio.

Decisiones Pendientes

- ☐ Confirmar arquitectura general.
- ☐ Definir System Architecture.
- ☐ Diseñar Information Flow.
- ☐ Diseñar Conceptual Data Model.
- ☐ Formalizar Governance.
- ☐ Completar ADRs.

Próxima Fase

05. System Architecture

Objetivo: Diseñar la arquitectura conceptual del POS definiendo motores, componentes, límites, responsabilidades e interacción entre ellos.

Estado de Madurez

Área	Madurez
Foundation	Alta
Dominio	Alta
Lenguaje	Alta
Modelo Conceptual	Media
Arquitectura	Baja
Información	Baja
Datos	Baja
Implementación	Nula

Próximo Hito

Finalizar la **System Architecture v0.1** y validar que el modelo conceptual soporte la evolución del sistema sin depender de ninguna tecnología específica.

Principios Fundamentales Descubiertos

- ✓ El POS organiza transformaciones.
- ✓ El trabajo evoluciona hacia capacidades.
- ✓ Las entidades existen para transformarse.
- ✓ Los procesos generan cambio.
- ✓ El conocimiento es información validada y reutilizable.
- ✓ Los activos representan valor acumulado.
- ✓ La arquitectura precede a la implementación.
- ✓ Las herramientas implementan el sistema; no lo definen.

Resumen Ejecutivo

El proyecto se encuentra en una etapa de consolidación del dominio. Ya existe una filosofía definida, un lenguaje ubicuo consistente y un modelo conceptual inicial. Las siguientes fases estarán orientadas a transformar ese conocimiento en una arquitectura formal, preparada para soportar múltiples implementaciones sin perder coherencia ni independencia tecnológica.